



A Short Demo Article: Regression Models for Count Data in R

Achim Zeileis 
Universität Innsbruck

Second Author
Plus Affiliation

Abstract

This short article illustrates how to write a manuscript for the *Journal of Statistical Software* (JSS) using its $\text{\LaTeX}$ style files. Generally, we ask to follow JSS's style guide and FAQs precisely. Also, it is recommended to keep the $\text{\LaTeX}$ code as simple as possible, i.e., avoid inclusion of packages/commands that are not necessary. For outlining the typical structure of a JSS article some brief text snippets are employed that have been inspired by `?`, discussing count data regression in R. Editorial comments and instructions are marked by vertical bars.

Keywords: JSS, style guide, comma-separated, not capitalized, R.

1. Introduction to LSSPCA

1.1. Data

X is an $n \times p$ matrix with columns centered to mean zero.

The objective is to compute sparse principal components (sPCs), that is, linear combinations of only a few of the variables.

Let $S = X^\top X$ be the covariance matrix. Let $v_1, \dots, v_p$ ($\|v_j\| = 1$, $j = 1, \dots, p$) be the eigenvectors of S , ordered by decreasing corresponding eigenvalues, $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p$. In SPCA literature, the v -vectors are called the *loadings* of the PCs. The principal components (PCs) are defined as $y_j = Xv_j$.

The sparse loadings are the vectors $a_j = (a_{i1}, \dots, a_{ip})$, $\|a_j\| = 1$, $j = 1, \dots, r$, each of which has only $c_j \leq p$ non-zero elements. The quantity c_j is called the *cardinality* of a_j . The sPCs are then equal to $t_j = Xa_j$.

1.2. Least squares Sparse Principal Component Analysis (LSSPCA)

LSSPCA solutions are derived using two different objectives.

Objective 1 (max VEXP)

$$\min_{a_j} \|X - t_j b^\top\|^2, \quad \text{subject to} \quad \text{card}(a_j) < p, \quad a_j^\top S a_i = 0, \quad i < j. \quad (1)$$

Objective 2 (max cor PCs)

$$\max_{a_j} \text{vert cor}(t_j, y_j), \quad \text{subject to} \quad a_j^\top S a_i = 0, \quad i < j, \quad \text{card}(a_j) < p. \quad (2)$$

We now face two challenges. The first is to derive the solutions when the indices of the non-zero loadings are known. The second is to identify the optimal set of these indices for each sPC.

We begin by assuming that these indices are known. Let ind_j denote the set of indices for the j th sPC, and let a dot above a symbol denote the operation of retaining the elements of a vector or the columns of a matrix corresponding to these indices. In R syntax, we can write $\dot{a}_j = a_j[\text{ind}_j]$ and $\dot{S}_j = S[, \text{ind}_j]$ and $t_j = \dot{X} \dot{a}_j$.

Objective 1: max variance explained

It can be shown that the solution to Objective 1 requires solving

$$\max_{a_j} \frac{a_j^\top X^\top X X^\top X a_j}{a_j^\top X^\top X a_j} \equiv \max_{a_j} \frac{a_j^\top S S a_j}{a_j^\top S a_j}, \quad a_j^\top S a_i = 0, \quad i < j, \quad \text{card}(a_j) < p. \quad (3)$$

The objective function ?? is the *variance explained* (VEXP) by the j th component. Without cardinality constraints it is the objective of standard PCA.

In passing, we note that, without cardinality constraints, objective function ?? is original PCA objective formulated by ?, which has solutions

$$\max_{v_j} a_j^\top S a_j, \quad a_j^\top S a_i = 0, \quad i < j.$$

These solutions are ? definition of PCA objective. The equivalence between the two objectives is no longer valid when cardinality constraints are introduced (?).

Uncorrelated LSSPCA (uSPCA). The exact solutions to objective ?? are computed by adding the constraints directly into the variance explained expression by means of the generic $\dot{X}$ matrix. Hence the optimal sparse loadings, $\dot{a}_j$, are the generalized eigenvectors corresponding to the largest eigenvalue, λ_j , solution to

$$C_j \dot{S}_j^\top \dot{S}_j \dot{a}_j = \lambda_j \dot{X}_j^\top \dot{X}_j \dot{a}_j, \quad (4)$$

where the matrices C_j ($C_1 = I$) enforce the orthogonality constraints.

The orthogonality constraints require that the cardinality of the solution loadings satisfy $c_j \geq j$. Furthermore, the solutions can be computationally demanding and unstable.

Correlated LSSPCA (cSPCA). Approximate but simpler to compute solutions can be obtained from the orthogonal residuals of the previously computed sPCs. Let

$$Q_j = \left(I_p - \frac{t_{j-1}t_{j-1}^\top}{t_{j-1}^\top t_{j-1}} \right) Q_{j-1} = H_j Q_{j-1}, \quad j = 2, 3, \dots, r, \quad (5)$$

where H_j is the *hat* matrix and $Q_1 = X$. The matrices Q_j are often called *deflated* matrices. Note that the use of the deflated matrices is a direct consequence of the Courant–Fischer min–max theorem and is typically used in the power method for computing the eigendecomposition. Then, the cSPCA solution loadings are obtained as the generalized eigenvectors corresponding to the largest eigenvalue:

$$\dot{Q}_j^\top X X^\top \dot{Q}_j \dot{a}_j = \mu_j \dot{X}_j^\top \dot{X}_j \dot{a}_j. \quad (6)$$

The first solution is equal to the first uSPCA solution but, in general, the following ones are not equal to the corresponding uSPCA solutions. The cSPCA solutions typically show very mild correlation with one another, with increasing values for sPCs of higher order. However, their cardinality is not constrained by the uncorrelatedness requirements and, in most cases, it is lower than their order.

$$\mu_j = \max_{a_j} \frac{a_j^\top X^\top Q_j Q_j^\top X a_j}{a_j^\top X^\top X a_j}. \quad (7)$$

This is an approximate measure of the variance explained. Because the resulting PCs are correlated, the exact VEXP must be obtained as the difference between the cumulative VEXP (CVEXP) explained by the first j csPCs and that explained by the preceding $(j - 1)$ csPCs:

$$\text{VEXP}(t_j) = \text{trace}\left(X^\top T_j (T_j^\top T_j)^{-1} T_j^\top X\right) - \text{trace}\left(X^\top T_{j-1} (T_{j-1}^\top T_{j-1})^{-1} T_{j-1}^\top X\right), \quad (8)$$

where T_j is the matrix with columns equal to the first j sPCs.

Objective 2: max correlation with the PCs

The sPCs obtained by maximizing the variance explained are usually highly correlated with the corresponding PCs. However, as the order of the components increases, these correlations tend to decrease. A different approach to computing sPCs is Projection LSSPCA (pSPCA). In this approach, the sPCs are obtained by projecting the residual PCs onto the x -variables. Let $z_j = Q_j v_j$ denote the first PC of the deflated matrix Q_j . Then the pSPCA solution loadings are obtained as the regression coefficients, standardized to unit norm, from the regression of z_j onto $\dot{X}_j$:

$$\dot{a}_j \propto (\dot{X}_j^\top \dot{X}_j)^{-1} \dot{X}_j^\top z_j. \quad (9)$$

The psPCs have maximum multiple squared correlation (the R^2 coefficient) for the given set of indices ind_j .

1.3. Variable selection

Variable selection is notoriously a computationally demanding, NP-hard task that requires greedy algorithms for its solution.

Regression-based variable selection algorithms are used to determine the pSPCA solution. Thus, we implemented standard forward, backward, and forward stepwise variable selection algorithms used for regression, with a stopping rule based on the current R^2 . Because the response variable z_j is proportional to an eigenvector of Q_j , the algorithms are simplified using the fact that $z_j^\top X = z_j^\top Q_j = v_j \lambda_j$. These algorithms have approximate complexity $O(p^2)$.

For uSPCA and cSPCA we implemented forward and backward algorithms that terminate after a minimal proportion of the CVEXP of the corresponding number of PCs is reached or, respectively, not passed. These algorithms use the approximate VEXP in Equation ?? to determine the variables to select or eliminate and are computationally very expensive. The forward selection is approximately $O(pk^4)$ for selecting k variables, even when VEXP is computed with the power method to save computation time. However, a dramatic reduction in computational complexity can be achieved by selecting the variables using one of the algorithms implemented for pSPCA and computing the CVEXP at each iteration, which is then used as the stopping rule. While the exact cSPCA-based algorithm may yield more efficient solutions, in many cases there is little difference from the results obtained using the regression variable selection algorithms.

2. Computational details

Since uSPCA and cSPCA require solving generalized eigenvector problems, the computation for large matrices is computationally demanding. The problem is even more taxing in variable selection. Therefore, we put some care to make the computation feasible on large matrices.

The major expedient that we use is to do the demanding $O(np^2)$ multiplication to compute the covariance matrix $S = X^\top X$ only once at the beginning. Then deflations are done with cheap rank-1 updates as $Q_{j+1} = Q_j - \frac{t_j t_j^\top}{t_j^\top t_j}$

Let

$$S_j = Q_j^\top Q_j, \quad M_j = S_j S_j,$$

and let a_j be the cSPCA loading vector solving

$$M_j a_j = S_j S_j a_j = \mu_j S a_j,$$

where $S = X^\top X$ is the original matrix and μ_j is the largest generalized eigenvalue. The score vector is always

$$t_j = X a_j,$$

so

$$t_j^\top t_j = a_j^\top S a_j.$$

With

$$Q_{j+1} = \left(I - \frac{t_j t_j^\top}{t_j^\top t_j} \right) Q_j,$$

define

$$g_j = Q_j^\top t_j, \quad d_j = t_j^\top t_j = a_j^\top S a_j.$$

Then

$$S_{j+1} = Q_{j+1}^\top Q_{j+1} = S_j - \frac{g_j g_j^\top}{d_j},$$

which is a rank-1 downdate. Consequently,

$$M_{j+1} = S_{j+1}S_{j+1}^\top = M_j - \frac{S_j g_j g_j^\top + g_j g_j^\top S_j}{d_j} + \frac{g_j^\top g_j}{d_j^2} g_j g_j^\top.$$

Thus M_j is updated by a sum of rank-1 terms, avoiding recomputation of M_{j+1} from scratch. Hence the update of M_j is a sum of rank-1 terms.

The generalized eigenvalue relation is consistent with these definitions, since

$$a_j^\top M_j a_j = a_j^\top S_j S_j a_j = \mu_j a_j^\top S a_j = \mu_j t_j^\top t_j,$$

where again μ_j is the largest generalized eigenvalue. This identity is useful for interpreting the scalar criterion associated with a_j , while the actual deflation formulas above are expressed in terms of $g_j = Q_j^\top t_j$ and $d_j = t_j^\top t_j = a_j^\top S a_j$.

Using these rank-1 updates gives a substantial computational saving compared with recomputing the matrices from the deflated data matrix at each step. Recomputing the covariance matrix S_j from Q_j costs $O(np^2)$, whereas updating it directly costs only $O(p^2)$. For M_j , the saving is larger: recomputing it requires multiplying two $p \times p$ matrices, which costs $O(p^3)$, whereas the corresponding update uses only matrix-vector products and outer products, so its cost is $O(p^2)$. The reduction is therefore substantial, especially when p is large.

3. JSS EXAMPLE

4. Introduction: Count data regression in R

The introduction is in principle “as usual”. However, it should usually embed both the implemented *methods* and the *software* into the respective relevant literature. For the latter both competing and complementary software should be discussed (within the same software environment and beyond), bringing out relative (dis)advantages. All software mentioned should be properly `\cite{}`d. (See also Appendix ?? for more details on `BIBTEX`.)

For writing about software JSS requires authors to use the markup `\proglang{}` (programming languages and large programmable systems), `\pkg{}` (software packages), `\code{}` (functions, commands, arguments, etc.). If there is such markup in (sub)section titles (as above), a plain text version has to be provided in the `LATEX` command as well. Below we also illustrate how abbreviations should be introduced and citation commands can be employed. See the `LATEX` code for more details.

Modeling count variables is a common task in economics and the social sciences. The classical Poisson regression model for count data is often of limited use in these disciplines because empirical count data sets typically exhibit overdispersion and/or an excess number of zeros. The former issue can be addressed by extending the plain Poisson regression model in various directions: e.g., using sandwich covariances or estimating an additional dispersion parameter (in a so-called quasi-Poisson model). Another more formal way is to use a negative binomial (NB) regression. All of these models belong to the family of generalized linear models (GLMs).

However, although these models typically can capture overdispersion rather well, they are in many applications not sufficient for modeling excess zeros. Since ? there is increased interest in zero-augmented models that address this issue by a second model component capturing zero counts. An overview of count data models in econometrics, including hurdle and zero-inflated models, is provided in ?.

In R (?), GLMs are provided by the model fitting functions `glm()` in the **stats** package and `glm.nb()` in the **MASS** package (?, Chapter 7.4) along with associated methods for diagnostics and inference. The manuscript that this document is based on (?) then introduced hurdle and zero-inflated count models in the functions `hurdle()` and `zeroinfl()` in the **pscl** package (?). Of course, much more software could be discussed here, including (but not limited to) generalized additive models for count data as available in the R packages **mgcv** ?, **gamlss** (?), or **VGAM** (?).

5. Models and software

The basic Poisson regression model for count data is a special case of the GLM framework ?. It describes the dependence of a count response variable y_i ($i = 1, \dots, n$) by assuming a Poisson distribution $y_i \sim \text{Pois}(\mu_i)$. The dependence of the conditional mean $E[y_i | x_i] = \mu_i$ on the regressors x_i is then specified via a log link and a linear predictor

$$\log(\mu_i) = x_i^\top \beta, \quad (10)$$

where the regression coefficients β are estimated by maximum likelihood (ML) using the iterative weighted least squares (IWLS) algorithm.

Note that around the `{equation}` above there should be no spaces (avoided in the L^AT_EX code by % lines) so that “normal” spacing is used and not a new paragraph started.

R provides a very flexible implementation of the general GLM framework in the function `glm()` (?) in the **stats** package. Its most important arguments are

```
glm(formula, data, subset, na.action, weights, offset,
    family = gaussian, start = NULL, control = glm.control(...),
    model = TRUE, y = TRUE, x = FALSE, ...)
```

where `formula` plus `data` is the now standard way of specifying regression relationships in R/S introduced in ?. The remaining arguments in the first line (`subset`, `na.action`, `weights`, and `offset`) are also standard for setting up formula-based regression models in R/S. The arguments in the second line control aspects specific to GLMs while the arguments in the last line specify which components are returned in the fitted model object (of class ‘`glm`’ which inherits from ‘`lm`’). For further arguments to `glm()` (including alternative specifications of starting values) see `?glm`. For estimating a Poisson model `family = poisson` has to be specified.

As the synopsis above is a code listing that is not meant to be executed, one can use either the dedicated `{Code}` environment or a simple `{verbatim}` environment for this. Again, spaces before and after should be avoided.

Type	Distribution	Method	Description
GLM	Poisson	ML	Poisson regression: classical GLM, estimated by maximum likelihood (ML)
		Quasi	“Quasi-Poisson regression”: same mean function, estimated by quasi-ML (QML) or equivalently generalized estimating equations (GEE), inference adjustment via estimated dispersion parameter
		Adjusted	“Adjusted Poisson regression”: same mean function, estimated by QML/GEE, inference adjustment via sandwich covariances
	NB	ML	NB regression: extended GLM, estimated by ML including additional shape parameter
Zero-augmented	Poisson	ML	Zero-inflated Poisson (ZIP), hurdle Poisson
	NB	ML	Zero-inflated NB (ZINB), hurdle NB

Table 1: Overview of various count regression models. The table is usually placed at the top of the page (`[t!]`), centered (`\centering`), has a caption below the table, column headers and captions are in sentence style, and if possible vertical lines should be avoided.

Finally, there might be a reference to a `{table}` such as Table `??`. Usually, these are placed at the top of the page (`[t!]`), centered (`\centering`), with a caption below the table, column headers and captions in sentence style, and if possible avoiding vertical lines.

6. Illustrations

For a simple illustration of basic Poisson and NB count regression the `quine` data from the **MASS** package is used. This provides the number of `Days` that children were absent from school in Australia in a particular year, along with several covariates that can be employed as regressors. The data can be loaded by

```
R> data("quine", package = "MASS")
```

and a basic frequency distribution of the response variable is displayed in Figure `??`.

For code input and output, the style files provide dedicated environments. Either the “agnostic” `{CodeInput}` and `{CodeOutput}` can be used or, equivalently, the environments `{Sinput}` and `{Soutput}` as produced by `Sweave()` or **knitr** when using the `render_sweave()` hook. Please make sure that all code is properly spaced, e.g., using `y = a + b * x` and *not* `y=a+b*x`. Moreover, code input should use “the usual” command prompt in the respective software system. For R code, the prompt `"R> "` should be used with `"+"` as the continuation prompt. Generally, comments within the code chunks should be avoided – and made in the regular $\text{\LaTeX}$ text instead. Finally, empty lines before and after code input/output should be avoided (see above).

As a first model for the `quine` data, we fit the basic Poisson regression model. (Note that JSS prefers when the second line of code is indented by two spaces.)

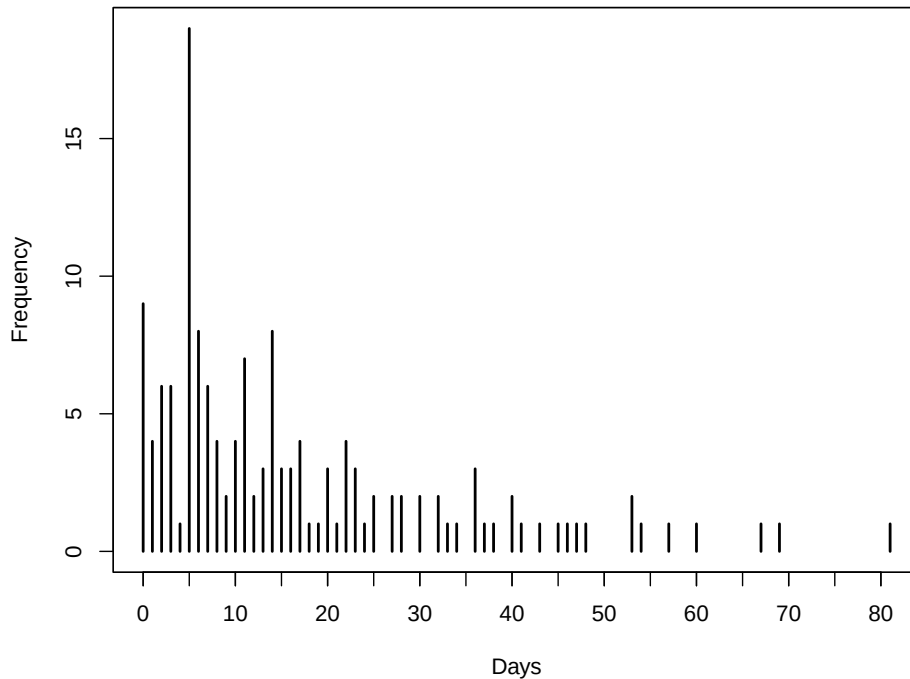


Figure 1: Frequency distribution for number of days absent from school.

```
R> m_pois <- glm(Days ~ (Eth + Sex + Age + Lrn)^2, data = quine,
+   family = poisson)
```

To account for potential overdispersion we also consider a negative binomial GLM.

```
R> library("MASS")
R> m_nbin <- glm.nb(Days ~ (Eth + Sex + Age + Lrn)^2, data = quine)
```

In a comparison with the BIC the latter model is clearly preferred.

```
R> BIC(m_pois, m_nbin)
```

	df	BIC
m_pois	18	2046.851
m_nbin	19	1157.235

Hence, the full summary of that model is shown below.

```
R> summary(m_nbin)
```

Call:

```
glm.nb(formula = Days ~ (Eth + Sex + Age + Lrn)^2, data = quine,
       init.theta = 1.60364105, link = log)
```

Coefficients: (1 not defined because of singularities)

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	3.00155	0.33709	8.904	< 2e-16 ***
EthN	-0.24591	0.39135	-0.628	0.52977
SexM	-0.77181	0.38021	-2.030	0.04236 *
AgeF1	-0.02546	0.41615	-0.061	0.95121
AgeF2	-0.54884	0.54393	-1.009	0.31296
AgeF3	-0.25735	0.40558	-0.635	0.52574
LrnSL	0.38919	0.48421	0.804	0.42153
EthN:SexM	0.36240	0.29430	1.231	0.21818
EthN:AgeF1	-0.70000	0.43646	-1.604	0.10876
EthN:AgeF2	-1.23283	0.42962	-2.870	0.00411 **
EthN:AgeF3	0.04721	0.44883	0.105	0.91622
EthN:LrnSL	0.06847	0.34040	0.201	0.84059
SexM:AgeF1	0.02257	0.47360	0.048	0.96198
SexM:AgeF2	1.55330	0.51325	3.026	0.00247 **
SexM:AgeF3	1.25227	0.45539	2.750	0.00596 **
SexM:LrnSL	0.07187	0.40805	0.176	0.86019
AgeF1:LrnSL	-0.43101	0.47948	-0.899	0.36870
AgeF2:LrnSL	0.52074	0.48567	1.072	0.28363
AgeF3:LrnSL	NA	NA	NA	NA

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for Negative Binomial(1.6036) family taken to be 1)

Null deviance: 235.23 on 145 degrees of freedom
 Residual deviance: 167.53 on 128 degrees of freedom
 AIC: 1100.5

Number of Fisher Scoring iterations: 1

Theta: 1.604
 Std. Err.: 0.214

2 x log-likelihood: -1062.546

7. Summary and discussion

■ As usual ...

Computational details

If necessary or useful, information about certain computational details such as version numbers, operating systems, or compilers could be included in an unnumbered section. Also, auxiliary packages (say, for visualizations, maps, tables, ...) that are not cited in the main text can be credited here.

The results in this paper were obtained using R .4 with the **MASS** 7.3.60.0.1 package. R itself and all packages used are available from the Comprehensive R Archive Network (CRAN) at <https://CRAN.R-project.org/>.

Acknowledgments

All acknowledgments (note the AE spelling) should be collected in this unnumbered section before the references. It may contain the usual information about funding and feedback from colleagues/reviewers/etc. Furthermore, information such as relative contributions of the authors may be added here (if any).

References

A. More technical details

Appendices can be included after the bibliography (with a page break). Each section within the appendix should have a proper section title (rather than just *Appendix*).

For more technical style details, please check out JSS's style FAQ at <https://www.jstatsoft.org/pages/view/style#frequently-asked-questions> which includes the following topics:

- Title vs. sentence case.
- Graphics formatting.
- Naming conventions.
- Turning JSS manuscripts into R package vignettes.
- Trouble shooting.
- Many other potentially helpful details...

B. Using Bib_TE_X

References need to be provided in a Bib_TE_X file (`.bib`). All references should be made with `\cite`, `\citet`, `\citep`, `\citealp` etc. (and never hard-coded). These commands yield different formats of author-year citations and allow to include additional details (e.g., pages, chapters, ...) in brackets. In case you are not familiar with these commands see the JSS style FAQ for details.

Cleaning up Bib_TE_X files is a somewhat tedious task – especially when acquiring the entries automatically from mixed online sources. However, it is important that informations are complete and presented in a consistent style to avoid confusions. JSS requires the following format.

- JSS-specific markup (`\proglang`, `\pkg`, `\code`) should be used in the references.
- Titles should be in title case.
- Journal titles should not be abbreviated and in title case.
- DOIs should be included where available.
- Software should be properly cited as well. For R packages `citation("pkgname")` typically provides a good starting point.

Affiliation:

Achim Zeileis

Journal of Statistical Software

and

Department of Statistics

Faculty of Economics and Statistics

Universität Innsbruck

Universitätsstr. 15

6020 Innsbruck, Austria

E-mail: Achim.Zeileis@R-project.org

URL: <https://www.zeileis.org/>